



CYBERSECURITY DEPARTMENT — INTERNSHIP PROJECT

Design, Implementation, and Monitoring of an Inline Intrusion Prevention System Using Snort

Project Documentation

From network topology design to a working, monitored intrusion prevention lab with a live alert dashboard

Prepared by
Guellil Imane
Rezzag Nadjia
Hachani Chaima

Department: Cybersecurity
Supervisor: Mr. Mustafa, Mr. Djamil

Table of Contents

1. Introduction	3
2. System Architecture Overview	3
3. Phase 1 — Virtual Machine Setup	4
4. Phase 2 — Network Configuration	5
5. Theoretical Background	5
6. Lab Architecture and Setup	6
7. Detection Rule Development.....	7
8. Dashboard Implementation.....	9
9. Testing and Results	11
10. Future Work.....	12
11. Conclusion	12

1. Introduction

This document records the complete design and build process of a small-scale Intrusion Prevention System (IPS) laboratory, carried out as part of a cybersecurity internship. The objective was to gain hands-on, end-to-end experience deploying Snort as an inline network security control, a system that inspects live traffic in real time and actively blocks malicious requests, rather than only alerting on them after the fact.

The lab simulates a simplified but realistic environment: an attacker/client machine (Kali Linux) generates both legitimate and malicious traffic, a dedicated Snort IPS virtual machine sits inline between two network segments, which inspects every packet that attempts to cross against a custom rule set and either forwards it or drops it, and an Ubuntu Server virtual machine represents the protected asset. A lightweight, browser-based alert dashboard was subsequently built on top of Snort's alert log to make ongoing monitoring practical rather than requiring manual log inspection over SSH.

This document walks through the project in the order it was actually carried out.

1.1 Objectives

- Deploy Snort as a true inline Intrusion Prevention System (IPS), capable of dropping malicious packets before they reach the protected host, not merely alerting on them.
- Design and configure a two-subnet routed network topology that forces all inter-segment traffic through the inspection point.
- Write and validate custom detection rules targeting a specific, well-understood attack class SQL injection and confirm they trigger correctly under live attack traffic.
- Build a practical, real-time monitoring interface on top of Snort's alert output, without introducing unnecessary infrastructure such as a database.

2. System Architecture Overview

The lab consists of three virtual machines connected across two isolated internal networks, plus a NAT interface on each machine used only for internet access during setup (package installation, source downloads). No traffic relevant to the security testing traverses the NAT interfaces.

2.1 Hosts and Roles

Host	Role	Key Interfaces / IPs
Kali (attacker/client)	Generates both legitimate traffic and simulated attacks against the protected server.	Net A: 10.0.0.2/24
Snort VM	Intrusion Prevention System. Bridges Net A and Net B; every packet crossing between the two subnets is inspected inline before being forwarded.	Net A: 10.0.0.1/24 Net B: 10.0.1.2/24

Ubuntu Server (target)	Protected asset. Runs a lightweight web service that receives the attacker's requests.	Net B: 10.0.1.2/24
------------------------	--	--------------------

2.2 Network Segments

- Net A (10.0.0.0/24): connects Kali to the Snort VM. This is the only path Kali's traffic can take toward the server.
- Net B (10.0.1.0/24): connects the Snort VM to the Ubuntu Server. Only traffic that Snort has allowed through ever reaches this segment.
- NAT: a per-VM adapter used only for internet access during setup (apt package installs, source downloads). Not part of the inspected data path.

Because the Snort VM is the only device connected to both Net A and Net B, it physically cannot be bypassed: any packet travelling between the attacker and the server must pass through it. This routed, two subnet layout is what makes an inline deployment possible in the first place, and is discussed further in Section 6.

3. Phase 1: Virtual Machine Setup

3.1 Prerequisites

- Host machine with hardware virtualization enabled (VT-x/AMD-V) and sufficient free RAM for three concurrent VMs.
- VirtualBox installed on the host.
- Kali Linux ISO (attacker machine) and Ubuntu Server LTS ISO (Snort VM and target server).

3.2 Virtual Machine Specifications

VM Name	RAM	CPU	Disk	Network Adapters
Kali (attacker)	2 GB	2 cores	20 GB	2 (Net A, NAT)
Snort VM	4 GB	2 cores	30 GB	3 (Net A, Net B, NAT)
Ubuntu Server (target)	2 GB	2 cores	20 GB	2 (Net B, NAT)

3.3 VirtualBox Network Setup

Before starting the VMs, the following networks were created in VirtualBox so each adapter could be attached to the correct segment:

- Internal Network "net-a": used for Net A (Kali ↔ Snort VM).
- Internal Network "net-b": used for Net B (Snort VM ↔ Ubuntu Server).
- NAT: built into VirtualBox, attached on a separate adapter per VM, used only during setup.

4. Phase 2: Network Configuration

Each VM's internal-network interfaces were assigned static IP addresses using Netplan (the NAT interface uses DHCP). Interface names were confirmed on each machine before editing any configuration file, since default names vary between installations:

```
ip link show
```

A representative Netplan configuration for the Snort VM, which carries two internal interfaces, is shown below.

```
network:  version: 2   ethernets:
eth0:
# NAT — internet access      dhcp4: yes      eth1:
# Net A — toward Kali        dhcp4: no
addresses: [10.0.0.1/24]      eth2:          # Net
B — toward Ubuntu Server    dhcp4: no
addresses: [10.0.1.1/24]
```

Configuration was applied and verified on each machine with:

```
sudo netplan apply
ip addr ip route
```

4.1 Enabling IP Forwarding on the Snort VM

Since the Snort VM is the only path between Net A and Net B, the Linux kernel must be told to forward packets between its interfaces once they have cleared inspection. Without this, traffic never crosses from the attacker's side to the server's side, regardless of what the Snort rules say.

```
sudo sysctl -w net.ipv4.ip_forward=1 sudo bash -c 'echo
"net.ipv4.ip_forward=1" >> /etc/sysctl.conf' sudo sysctl -p
```

This setting is written to `/etc/sysctl.conf`, so — unlike the iptables rule discussed in Section 6 — it persists automatically across reboots.

5. Theoretical Background

5.1 IDS vs. IPS: Passive Alerting vs. Active Blocking

An Intrusion Detection System (IDS) operates passively: a copy of network traffic is inspected, and when a rule matches, an alert is logged for an analyst to review, but the original packet is never touched, and by the time the alert is written, the traffic has already reached its destination. An Intrusion Prevention System (IPS) goes a step further: it sits directly in the path of the traffic (inline), and for every packet it must make a decision, forward it or drop it, before that packet is allowed to continue. This distinction is the conceptual core of this project. Snort is capable of operating in either mode; this lab specifically targets true IPS behavior, meaning malicious packets never reach the protected server at all, rather than reaching it and merely being logged afterward.

5.2 How Snort Works

Snort's processing pipeline can be broken into three broad stages. First, the DAQ (Data Acquisition) layer is responsible for actually getting packets from the network interface into Snort's process, it is a pluggable abstraction, and which DAQ module is loaded determines whether Snort can only observe traffic or can also make a forward/drop decision on it (see Section 5.3). Second, the packet is decoded and preprocessed, Snort reassembles fragmented packets and reconstructed TCP streams so that an attack payload split across multiple packets is still matched correctly. Third, the rule-matching engine compares the packet (and, for HTTP traffic, the reassembled request) against the loaded rule set. Rules are evaluated for both protocol/header fields (source/destination IP and port, protocol) and content patterns within the payload. When a rule matches, Snort performs the action specified in that rule (alert, drop, or pass) and writes a corresponding entry to its log.

5.3 NFQUEUE and Why Inline Mode Requires It

Snort's default and simplest DAQ module, `afpacket`, works by placing the network interface into a mode where Snort receives a copy of every packet that passes through it. This is sufficient for building an IDS: Snort sees the traffic and can log or alert on it, but it has no mechanism to stop the original packet from continuing on its way, because it was only ever handed a copy. For a genuine IPS, Snort instead needs to be handed the actual packet and be given the authority to decide its fate.

This is exactly what the `NFQUEUE` DAQ module provides. On the Linux side, an `iptables` rule is used to divert matching packets, instead of letting the kernel route them immediately into a named `netfilter` queue. Snort, running with the `NFQUEUE` DAQ, reads packets out of that queue one at a time and must explicitly issue a verdict for each one: `NF_ACCEPT` (let the kernel continue routing it normally) or `NF_DROP` (discard it — the packet is never forwarded). This project therefore required rebuilding Snort's DAQ with `libnetfilter-queue` support (Section 6.4) and switching the runtime DAQ selection from `afpacket` to `nfq`, which is the single change that converts the lab from a passive sniffer into a true, active-blocking IPS.

5.4 SQL Injection as an Attack Class

SQL injection is an application-layer attack in which untrusted input is inserted into a database query without proper sanitization, allowing an attacker to alter the query's logic. Common variants include `UNION`-based injection (appending a second `SELECT` statement to extract data from unrelated tables), boolean/tautology-based injection (e.g. injecting a condition that is always true to bypass authentication or filtering logic), comment-based injection (using SQL comment syntax to truncate the rest of a legitimate query), and time-based blind injection (using a database function such as `SLEEP()` to infer information from response timing when no direct output is returned). This project's detection rules specifically target these four patterns, since they are well documented, easy to simulate safely in a lab, and produce a clear, recognizable signature in the request payload which makes them a good fit for network-layer, signature based detection with Snort. Section 7 details each rule and how it maps to one of these patterns.

6. Lab Architecture and Setup

6.1 Network Diagram

The topology is intentionally simple: two subnets joined only through the Snort VM, which acts simultaneously as the router and the inspection point between them.

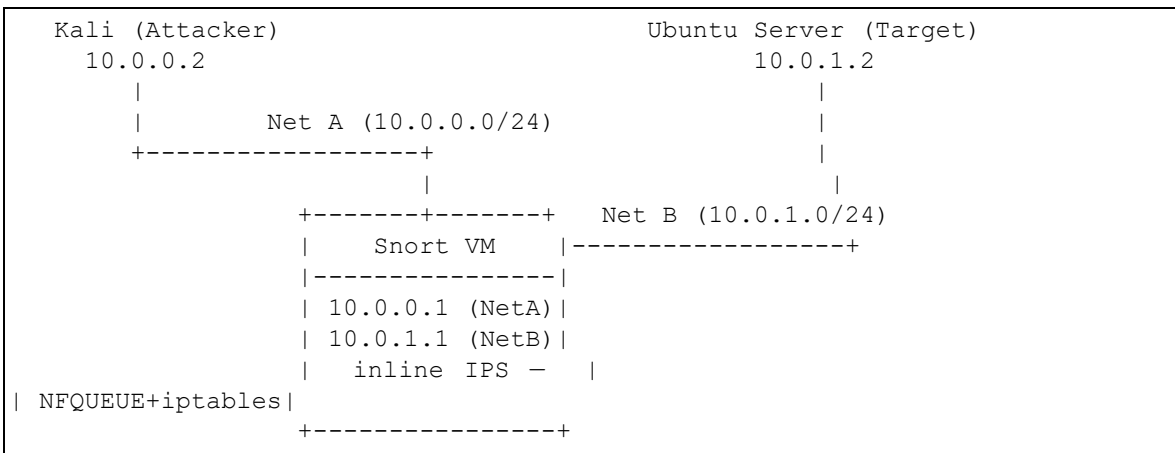


Figure 1 — Logical topology: two routed subnets bridged only by the Snort VM.

6.2 Why a Two-Subnet Routed Design

Placing the attacker and the target on the same broadcast domain as the Snort VM (a bridged/spanningport design) would let Snort observe traffic, but nothing would force that traffic to pass through it, a host could route around it. By instead making the Snort VM the only Layer-3 gateway between two separate subnets, every packet that needs to leave Net A for Net B (or vice-versa) has no path except through the Snort VM's routing and inspection logic. This is what turns "a machine that can see the traffic" into "a machine the traffic cannot avoid", the defining property of an inline IPS deployment.

6.3 Running Snort Inline

With the DAQ rebuilt and the iptables redirect in place, Snort is launched using the nfq DAQ, bound to queue 0, in inline (-Q) mode:

```
sudo /usr/local/snort/bin/snort --daq-dir /usr/local/lib/daq -Q --daq nfq \
--daq-var queue=0 -c /etc/snort/snort.conf -l /var/log/snort -A fast \ d -
u snort -g snort
```

Short now issues an explicit accept/drop verdict for every packet pulled from the queue, rather than only observing a copy of it — this is the command that puts the lab into active IPS mode, as opposed to passive alert-only monitoring.

7. Detection Rule Development

Detection was scoped to SQL injection specifically, since it is a single, well-defined attack class that maps cleanly onto the four patterns discussed in Section 5.4. Four custom rules were written into `/etc/snort/rules/local.rules`, each using the drop action so that, when running inline, a match results in the packet being discarded rather than merely logged.

7.1 Rule Set

```
# ----- SQL INJECTION (DROP) -----
drop tcp any any -> any 80 (msg:"SQL Injection - UNION SELECT Blocked";
content:"UNION"; http_uri; content:"SELECT"; http_uri; nocase;
sid:2001; rev:1;)

drop tcp any any -> any 80 (msg:"SQL Injection - OR Statement Blocked";
content:"' OR "; http_uri; nocase; sid:2002; rev:1;)

drop tcp any any -> any 80 (msg:"SQL Injection - Comment Bypass Blocked";
content:"--"; http_uri; nocase; sid:2003; rev:1;)

drop tcp any any -> any 80 (msg:"SQL Injection - Time-based Blind Blocked";
content:"SLEEP"; http_uri; nocase; sid:2004; rev:1;)
```

7.2 Rule-by-Rule Rationale and Test Traffic

Each rule was exercised from Kali using curl against the Ubuntu Server, with attack payloads passed as URL-encoded query-string values (raw literal spaces and quotes are rejected by curl's own parser, so %20 and %27 were used in place of a space and a single quote respectively).

SID 2001 — UNION SELECT

Targets the classic UNION-based injection pattern, where an attacker appends a second SELECT statement to pull data from an unrelated table (e.g. usernames and password hashes).

```
curl "http://10.0.1.2/?id=1%20UNION%20SELECT%20username,password%20FROM%20users"
```

SID 2002 — OR Statement

Targets tautology-based injection, where an always-true OR condition is used to bypass a WHERE clause or authentication check.

```
curl "http://10.0.1.2/?id=1%27%20OR%20%271%27=%271"
```

SID 2003 — Comment Bypass

Targets use of the SQL inline-comment sequence to truncate the remainder of a legitimate query after the attacker's injected clause.

```
curl "http://10.0.1.2/?id=1--"
```

SID 2004 — Time-based Blind

Targets time-based blind injection, where the SLEEP() function is used to infer information from response delay when the application returns no visible query output.

```
curl "http://10.0.1.2/?id=1%20SLEEP(5) "
```


7.3 Test Results Summary

SID	Attack Pattern Matched	Action	Test Result
2001	UNION SELECT — extracting data from another table	drop	Blocked as expected
2002	' OR ' tautology — bypassing a WHERE/auth clause	drop	Blocked after correcting URL encoding
2003	-- comment — truncating the rest of the query	drop	Blocked as expected
2004	SLEEP() — time-based blind injection	drop	Blocked as expected

All four rules fired correctly against their corresponding payload once the request reached the Snort VM, and — critically, per Section 9 — the matching packets were dropped rather than only logged, confirming inline blocking behavior rather than passive detection.

8. Dashboard Implementation

Snort's raw fast-alert log (`/var/log/snort/alert`) is plain text, one line per event, and reading it directly over SSH does not scale for ongoing monitoring, there is no aggregation, no filtering, and no visual sense of trends or volume. A lightweight dashboard was built on top of this log so that alerts are visible at a glance, without requiring manual log searching or any separate persistence layer.

8.1 Design Decisions

- Runs entirely on the Snort VM, since that is where the alert log already lives and where internet access (for installing Flask and loading Chart.js from a CDN) was already available.
- No database and no intermediate JSON storage: the log is parsed live from disk on every page load, so refreshing the page always reflects the current state of `/var/log/snort/alert`.
- Search and filtering run entirely client-side in JavaScript, against the full alert dataset embedded as a JSON array in the rendered page, this avoided the need for any backend query endpoints.

8.2 Technology Stack

Layer	Choice and Rationale

Backend	Python 3 with Flask, a lightweight framework requiring a single file and no build step, well suited to a small monitoring tool built under time pressure.
Frontend	Plain HTML/CSS with Chart.js loaded from a CDN for the bar and pie charts, no separate JavaScript framework or bundler was introduced.
Data source	Parsed directly from /var/log/snort/alert on every request; the alert-vs-drop action is read from Snort's own [Drop] log tag rather than looked up separately (see Section 8.4).

8.3 Features

- Search / filter bar: a free-text time filter, a protocol dropdown populated dynamically from whatever protocols actually appear in the current alert data, and an IP filter matching either source or destination, all evaluated client-side with no page reload.
- Bar chart: alert counts grouped by rule message, so the most frequently triggered rule is immediately visible.
- Pie chart: alert vs. drop breakdown, giving an at-a-glance sense of how much traffic was actively blocked versus only logged.
- Stat cards: total alert count, number of unique rule types triggered, number of distinct source IPs, and count of dropped (blocked) alerts.
- Paginated table: shows the 5 most recent alerts (Time, Message, SID, Protocol, Source, Destination, Action) with a "Show 5 more" button that reveals additional rows and disables itself once all matching rows are shown or a filter returns zero results.

8.4 Alert vs. Drop Detection

The first version of the parser determined whether an alert corresponded to a blocked packet by looking up its SID against /etc/snort/rules/local.rules. During testing it was noticed that Snort's own fast-alert log already tags dropped packets directly on the log line, for example:

```
08/26-10:46:39.562014 [Drop] [**] [1:2004:1] SQL Injection - Time-based
Blind Blocked [**] ...
```

This is a more accurate source of truth than the rules-file lookup, since it reflects what actually happened to that specific packet at runtime rather than the rule's statically configured action. The parser's regular expression was updated to capture this tag directly, with the rules-file lookup retained only as a fallback for alert-only rules that never receive a [Drop] tag.

```
r'^(?P<date>\d{2}/\d{2})-(?P<time>\d{2}:\d{2}:\d{2})\.\d+\s+'
r'(?:(?P<action_tag>\w+)\s+)?'
r'\[\\*\\*\\]\s+[(?P<gid>\d+):(P<sid>\d+):(P<rev>\d+)\s+]

```

8.5 Screenshots

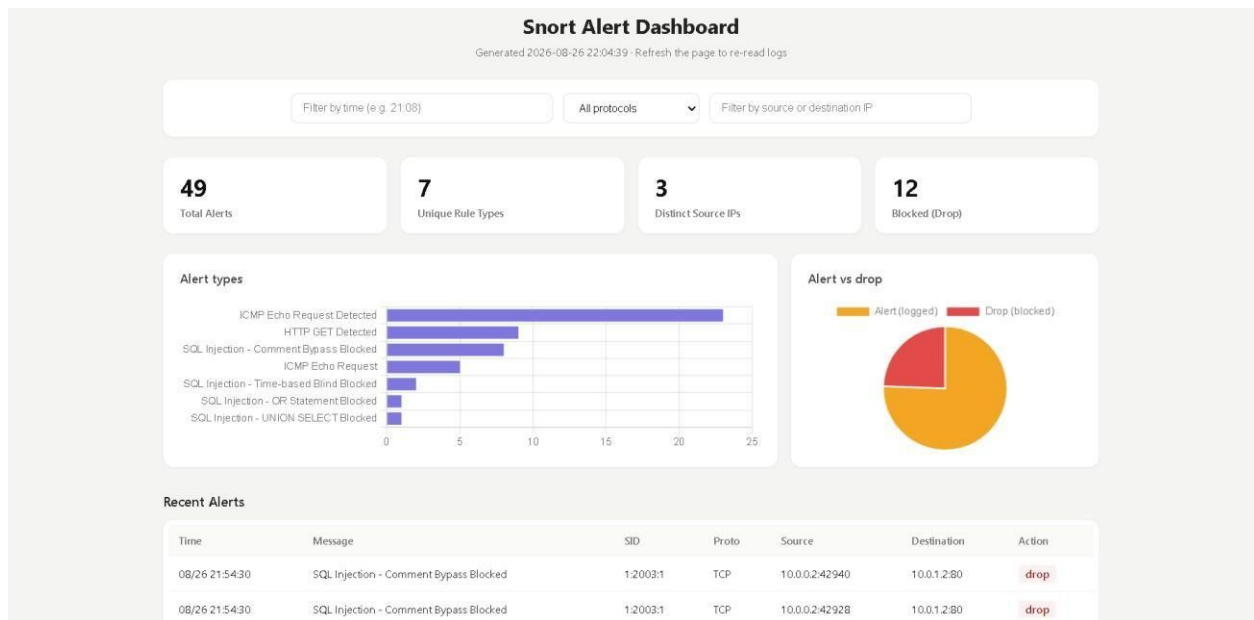


Figure 2 — Dashboard overview.

9. Testing and Results

9.1 Test Methodology

Each rule was tested by launching its corresponding attack payload from Kali toward the Ubuntu Server, then confirming three things in sequence: (1) the request reached the Snort VM and matched the intended rule, (2) the outcome recorded in the alert log matched the rule's configured action, and (3) for drop rules specifically, that the packet did not reach the server at all the defining behavior of an IPS as opposed to an IDS.

9.2 Confirming Packets Were Actually Dropped, Not Just Logged

Logging an alert is not, by itself, proof of inline blocking, a passive IDS produces identical log entries. The evidence that this deployment behaved as a true IPS came from two corroborating sources: first, the explicit [Drop] tag written by Snort itself into the fast-alert log at the moment of the verdict (Section 8.4), confirming that Snort issued an NF_DROP verdict to the kernel for that packet rather than NF_ACCEPT; and second, the corresponding curl request on the Kali side for a drop-matched payload did not receive a normal HTTP response from the server, consistent with the packet never having been forwarded past the Snort VM. Together, these confirm the SQL injection payloads were blocked in transit rather than merely observed and recorded after delivery.

9.3 Results Table

SID	Rule	Action	Confirmed Firing?
-----	------	--------	-------------------

2001	SQL Injection - UNION SELECT Blocked	drop	Yes
2002	SQL Injection - OR Statement Blocked	drop	Yes (after URLencoding fix)
2003	SQL Injection - Comment Bypass Blocked	drop	Yes
2004	SQL Injection - Time-based Blind Blocked	drop	Yes

Baseline alert-only rules (covering ICMP, SSH connection attempts, and general HTTP activity) were also present in the rule set and used to validate that the protocol dropdown and general alerting pipeline in the dashboard scaled correctly beyond a single protocol, independent of the SQL injection drop rules that are the focus of this project.

10. Future Work

- YARA file-scanning integration: static detection rules (an initial signature set covering the EICAR test string, PHP web-shell indicators, and common obfuscation patterns) have already been written; the remaining work is wiring a file-scan section into the existing dashboard alongside the Snort alert feed. This was deliberately scoped out of the initial dashboard build so the Snort-only pipeline could be completed and verified first, and is currently in progress.
- Expanding rule coverage beyond SQL injection to other common web attack classes (directory traversal, cross-site scripting, command injection).
- Adding persistence for Snort and the dashboard via systemd services, and persisting the NFQUEUE iptables rule, to remove the manual restart checklist documented, particularly important to avoid any risk of the lab breaking during a live demonstration.
- Forwarding high-severity alerts to email or Slack in addition to the dashboard, so that blocking events do not require someone to be actively watching the page.
- Completing validation of protocol coverage beyond TCP and ICMP (e.g. a UDP/DNS test rule) to confirm the dashboard's protocol filter continues to scale correctly as new rule types are added.

11. Conclusion

This project set out to build a working inline Intrusion Prevention System using Snort, backed by custom detection rules and a practical monitoring interface, rather than a passive, alert-only IDS. That objective was met: a two-subnet routed lab was built in which the Snort VM cannot be bypassed, Snort was moved from its default afpacket DAQ to the NFQUEUE DAQ so it could issue real accept/drop verdicts on live traffic, four SQL injection detection rules were written and verified to correctly block their corresponding attack payloads, and a Flask-based dashboard was built on top of Snort's own alert log to make the resulting activity practical to monitor without manual log reading.

Beyond the working system itself, the project produced a concrete, transferable set of skills: Linux network configuration and routing across multiple interfaces, compiling and configuring Snort and its DAQ layer from source, writing and validating signature-based detection rules against real attack traffic, and building a small full-stack monitoring tool end to end. Just as importantly, the process of diagnosing real failures, a DAQ missing a required module, a non-persistent iptables rule silently breaking the pipeline after a reboot, stale processes masking a newly added rule, reflects the kind of troubleshooting judgment the internship was intended to develop, and is documented in full in Section 10 rather than smoothed over.